1. Implement menu-driven program to implement Stack using Array to perform all basic operations: PUSH, POP, PEEK, isEmpty, isFull, and DISPLAY

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 5

int top = -1;
int stack[SIZE];

int isEmpty()
{
   if (top == -1)
      return 1;
   return 0;
}
int isFull()
{
   if (top == SIZE - 1)
      return 1;
   return 0;
}

int main()
{
   int ch;
   do
   {
      printf(" ====> Stack Operations <==== \n");
      printf(" 1. Push\n 2. Pop\n 3. Peek\n 4. Display\n 5. Exit\n\n");
      printf("Enter Your Choice: ");
      scanf("%d", &ch);

      switch (ch)
      {
      case 1:
         if (isFull() == 1)
            printf("Stack is OverFlow ...\n");
         else
         {
            printf("Enter Your Value for Push: ");
            scanf("%d", &stack[++top]);
            printf("Element Push Successfully in Stack ....\n");
         }
         break;

      case 2:
```

```
            (isEmpty() == 1) ? printf("Stack is UnderFlow ...\n") : printf("Element Pop
Successfully in Stack ....\n");
            break;

        case 3:
            (isEmpty() == 1) ? printf("Stack is UnderFlow ...\n") : printf("%d is Peep in
Stack ....\n", stack[top]);
            break;

        case 4:
            if (isEmpty() == 1)
                printf("Stack is UnderFlow ...\n");
            else
            {
                for (int i = top; i >= 0; i--)
                    printf("%d\t", stack[i]);
                printf("\n");
            }
            break;

        case 5:
            printf("Exiting...\n");
            break;

        default:
            printf("Invalid Choice ... \n");
        }
    } while (ch != 5);
    return 0;
}
```

Output:

```
 ====> Stack Operations <====
 1. Push
 2. Pop
 3. Peek
 4. Display
 5. Exit

Enter Your Choice: 1
Enter Your Value for Push: 10
Element Push Successfully in Stack ....
 ====> Stack Operations <====
 1. Push
 2. Pop
 3. Peek
 4. Display
 5. Exit

Enter Your Choice: 1
Enter Your Value for Push: 20
Element Push Successfully in Stack ....
 ====> Stack Operations <====
 1. Push
 2. Pop
 3. Peek
 4. Display
 5. Exit

Enter Your Choice: 3
20 is Peep in Stack ....
 ====> Stack Operations <====
 1. Push
 2. Pop
 3. Peek
 4. Display
 5. Exit

Enter Your Choice: 2
Element Pop Successfully in Stack ....
 ====> Stack Operations <====
 1. Push
 2. Pop
 3. Peek
 4. Display
 5. Exit

Enter Your Choice: 4
20      10
 ====> Stack Operations <====
 1. Push
 2. Pop
 3. Peek
 4. Display
 5. Exit

Enter Your Choice: 5
Exiting...
```

2. Implement program to convert infix to postfix expression.

```
#include <stdio.h>
int main()
{
   char infix[200], postfix[200], stack[200];
   int top = -1, j = 0;

   printf("Enter Your Infix Expression: ");
   scanf("%[^\n]", infix);

   for (int i = 0; infix[i] != '\0'; i++)
   {
      switch (infix[i])
      {
      case '(':
         stack[++top] = infix[i];
         break;

      case ')':
         while (stack[top] != '(')
            postfix[j++] = stack[top--];
         stack[top--];
         break;

      case '^':
         while (stack[top] != '^')
            postfix[j++] = stack[top--];
         stack[++top] = infix[i];
         break;

      case '*':
      case '/':
         while (stack[top] == '^' || stack[top] == '*' || stack[top] == '/')
            postfix[j++] = stack[top--];
         stack[++top] = infix[i];
         break;

      case '+':
      case '-':
         while (stack[top] == '^' || stack[top] == '*' || stack[top] == '/' || stack[top] == '+'
|| stack[top] == '-')
            postfix[j++] = stack[top--];
         stack[++top] = infix[i];
         break;

      case ' ':
         continue;
```

```
            default:
                postfix[j++] = infix[i];
            }
        }

        while (stack[top] != -1)
        {
            if (stack[top] != '(')
                postfix[j++] = stack[top--];
            else
                stack[top--];
        }

        printf("Infix Expression: %s\n", infix);
        printf("Postfix Expression: %s\n\n", postfix);
        return 0;
}
```

Output:

```
Enter Your Infix Expression: a + b * c
Infix Expression: b + b * c
Postfix Expression: abc*+
```

3. Implement Program to Evaluate Prefix Expression.

```
#include <stdio.h>
#include <string.h>
int main()
{
   char prefix[200];
   int stack[200], top = -1, i, firstValue, secondValue;

   printf("Enter Your Prefix Expression: ");
   scanf("%[^\n]", prefix);

   for (i = strlen(prefix) - 1; i >= 0; i--)
   {
      if (prefix[i] == ' ')
         continue;

      if (prefix[i] >= '0' && prefix[i] <= '9')
         stack[++top] = prefix[i] - '0';
      else
      {
         firstValue = pop();
         secondValue = pop();

         switch (prefix[i])
         {
         case '+':
            stack[++top] = firstValue + secondValue;
            break;

         case '-':
            stack[++top] = firstValue - secondValue;
            break;

         case '*':
            stack[++top] = firstValue * secondValue;
            break;

         case '/':
            if (secondValue == 0)
            {
               printf("Division by Zero Error\n");
               return 0;
            }
            stack[++top] = firstValue / secondValue;
            break;

         case '^':
            firstValue = 1;
```

```
            for (int j = 0; j < secondValue; j++)
                firstValue = firstValue * secondValue;
            break;

        default:
            printf("Invalid Operator\n");
            return 0;
        }
    }
  }
  printf("Prefix Expression: %s\n", prefix);
  printf("Result: %d\n", stack[top--]);
  return 0;
}
```

Output:

```
Enter Your Prefix Expression: + 6 * 9 + 73
Prefix Expression: + 6 * 9 + 73
Result: 96
```

4. Implement menu-driven program to implement Simple Queue using Array to perform operations: INSERT (Enqueue), DELETE (Dequeue), PEEK, and DISPLAY.

```
#include <stdio.h>
int main()
{
   int front = -1, rear = -1, simpleQueue[5], ch;
   do
   {
      printf(" ====> Simple Queue Operations <==== \n");
      printf(" 1. Enqueue\n 2. Dequeue\n 3. Peek\n 4. Display\n 5. Exit\n\n");
      printf("Enter Your Choice: ");
      scanf("%d", &ch);

      switch (ch)
      {
      case 1:
         if (rear == 5 - 1)
            printf("Simple Queue is Empty ... \n");
         else
         {
            printf("Enter Your Value: ");
            scanf("%d", &simpleQueue[++rear]);
            front = 0;
            printf("%d is inserted succeffully ... \n", simpleQueue[rear]);
         }
         break;

      case 2:
         if (front == rear)
         {
            front = rear = -1;
            printf("Queue is Empty ... \n");
         }
         else
            printf("%d is deleted succeffully ... \n", simpleQueue[front++]);
         break;

      case 3:
         (front == -1) ? printf("Simple Queue is Empty ... \n") : printf("%d is peeked ...
\n", simpleQueue[front]);
         break;

      case 4:
         if (front == -1)
            printf("Simple Queue is Empty ... \n");
         else
         {
```

```
            for (int i = front; i <= rear; i++)
                printf("%d\t", simpleQueue[i]);
            printf("\n");
        }
        break;

    case 5:
        printf("Exiting...\n");
        break;

    default:
        break;
    }
  } while (ch != 5);
  return 0;
}
```

Output :

```
 ====> Simple Queue Operations <====
 1. Enqueue
 2. Dequeue
 3. Peek
 4. Display
 5. Exit

Enter Your Choice: 1
Enter Your Value: 10
10 is inserted succeffully ...
 ====> Simple Queue Operations <====
 1. Enqueue
 2. Dequeue
 3. Peek
 4. Display
 5. Exit

Enter Your Choice: 1
Enter Your Value: 20
20 is inserted succeffully ...
 ====> Simple Queue Operations <====
 1. Enqueue
 2. Dequeue
 3. Peek
 4. Display
 5. Exit

Enter Your Choice: 1
Enter Your Value: 30
30 is inserted succeffully ...
 ====> Simple Queue Operations <====
 1. Enqueue
 2. Dequeue
 3. Peek
 4. Display
 5. Exit

Enter Your Choice: 3
10 is peeked ...
 ====> Simple Queue Operations <====
 1. Enqueue
 2. Dequeue
 3. Peek
 4. Display
 5. Exit

Enter Your Choice: 2
10 is deleted succeffully ...
 ====> Simple Queue Operations <====
 1. Enqueue
 2. Dequeue
 3. Peek
 4. Display
 5. Exit

Enter Your Choice: 4
20      30
 ====> Simple Queue Operations <====
 1. Enqueue
 2. Dequeue
 3. Peek
 4. Display
 5. Exit

Enter Your Choice: 5
Exiting...
```

5. Implement menu-driven program to implement Circular Queue using Array to perform operations: INSERT (Enqueue), DELETE (Dequeue), and DISPLAY

```
#include <stdio.h>
int front = -1, rear = -1, circularQueue[5];

void enqueue()
{
   int value;

   printf("Enter Your Value: ");
   scanf("%d", &value);

   if (front == -1 && rear == -1)
   {
      front = rear = 0;
      printf("%d is inserted succefully ... \n", circularQueue[rear] = value);
   }

   else if (rear == front - 1)
      printf("Queue is Full ... \n");

   else if (front != 0)
   {
      if (rear == 5 - 1)
      {
         rear = 0;
         printf("%d is inserted succefully ... \n", circularQueue[rear] = value);
      }

      else
         printf("%d is inserted succefully ... \n", circularQueue[++rear] = value);
   }

   else if (rear == 5 - 1)
      printf("\n Queue Is Empty ... \n");

   else
      printf("%d is inserted succefully ... \n", circularQueue[++rear] = value);
}

int main()
{
   int ch;
   do
   {
      printf(" ====> Circular Queue Operations <==== \n");
      printf(" 1. Enqueue\n 2. Dequeue\n 3. Display\n 4. Exit\n\n");
```

```
        printf("Enter Your Choice: ");
        scanf("%d", &ch);

        switch (ch)
        {
        case 1:
           enqueue();
           break;

        case 2:
           (front == rear) ? printf("\n Queue Is Empty ... \n") : printf("\n %d Deleted
    Successfully ...\n", circularQueue[front++]);
           break;

        case 3:
           for (int i = front; i <= rear; i++)
              printf("%d \t", circularQueue[i]);
           printf("\n");
           break;

        case 4:
           printf("Exiting...\n");
           break;

        default:
           printf("You Have chhose Wrong Options");
           break;
        }
     } while (ch != 5);
     return 0;
  }
```

Output :

```
 ====> Circular Queue Operations <====
 1. Enqueue
 2. Dequeue
 3. Display
 4. Exit

Enter Your Choice: 1
Enter Your Value: 10
10 is inserted succefully ...
 ====> Circular Queue Operations <====
 1. Enqueue
 2. Dequeue
 3. Display
 4. Exit

Enter Your Choice: 1
Enter Your Value: 20
20 is inserted succefully ...
 ====> Circular Queue Operations <====
 1. Enqueue
 2. Dequeue
 3. Display
 4. Exit

Enter Your Choice: 3
10      20
 ====> Circular Queue Operations <====
 1. Enqueue
 2. Dequeue
 3. Display
 4. Exit

Enter Your Choice: 2

 10 Deleted Successfully ...
 ====> Circular Queue Operations <====
 1. Enqueue
 2. Dequeue
 3. Display
 4. Exit

Enter Your Choice: 4
Exiting...
```

9. Implement menu-driven program to perform Searching Techniques to search for an element in an array using both Linear Search and Binary Search algorithms.

```
#include <stdio.h>
int main()
{
   int choice, arr[5], value, found = 0, low = 0, high = 5 - 1, mid;
   do
   {
      printf("\n=====> Searching Techniques <=====\n");
      printf("1. Input Array\n 2. Linear Search\n 3. Binary Search\n 4. Display Array\n
5. Exit\n\n");
      printf("Enter Your Choice: ");
      scanf("%d", &choice);

      switch (choice)
      {
      case 1:
         printf("Enter %d Elements in Sorted Order:\n", 5);
         for (int i = 0; i < 5; i++)
            scanf("%d", &arr[i]);
         printf("Array Inserted Successfully\n");
         break;

      case 2:
         printf("Enter Value to Search: ");
         scanf("%d", &value);

         for (int i = 0; i < 5; i++)
            if (arr[i] == value)
            {
               printf("Value Found at Position %d\n", i + 1);
               found = 1;
               break;
            }
         if (found == 0)
            printf("Value Not Found\n");
         break;

      case 3:
         printf("Enter Value to Search: ");
         scanf("%d", &value);

         while (low <= high)
         {
            mid = (low + high) / 2;
            if (arr[mid] == value)
            {
```

```
                printf("Value Found at Position %d\n", mid + 1);
                found = 1;
                break;
            }
            else if (arr[mid] < value)
                low = mid + 1;
            else
                high = mid - 1;
        }
        if (found == 0)
            printf("Value Not Found\n");
        break;

    case 4:
        printf("Array Elements:\n");

        for (int i = 0; i < 5; i++)
            printf("%d\t", arr[i]);
        printf("\n");
        break;

    case 5:
        printf("Exiting...\n");
        break;

    default:
        printf("Invalid Choice\n");
    }
  } while (choice != 5);
  return 0;
}
```

Output :

```
====> Searching Techniques <====
 1. Input Array
 2. Linear Search
 3. Binary Search
 4. Display Array
 5. Exit

Enter Your Choice: 1
Enter 5 Elements in Sorted Order:
10 20 30 40 50
Array Inserted Successfully

====> Searching Techniques <====
 1. Input Array
 2. Linear Search
 3. Binary Search
 4. Display Array
 5. Exit

Enter Your Choice: 2
Enter Value to Search: 10
Value Found at Position 1

====> Searching Techniques <====
 1. Input Array
 2. Linear Search
 3. Binary Search
 4. Display Array
 5. Exit

Enter Your Choice: 3
Enter Value to Search: 40
Value Found at Position 4

====> Searching Techniques <====
 1. Input Array
 2. Linear Search
 3. Binary Search
 4. Display Array
 5. Exit

Enter Your Choice: 4
Array Elements:
10      20      30      40      50

====> Searching Techniques <====
 1. Input Array
 2. Linear Search
 3. Binary Search
 4. Display Array
 5. Exit

Enter Your Choice: 5
Exiting...
```